ARM PrimeCell Driver

SMC v1.1 (PL092)

API Specification

PL092-ISPC-1000-C

Open Access

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	C
PrimeCell Identifier	SMC
Code Version	1.1
PrimeCell Identifiers	PL092

Contents

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A	5, October, 2001	First issue of API
B	2, November, 2001	Issue of API version 1.0
C	22, January, 2002	Documentation correction to apSMC_Initialize

2 SUMMARY

The PrimeCell Static Memory Controller (SMC) is an Advanced Microcontroller Bus Architecture (AMBA) compliant System-on-Chip peripheral that is developed, tested and licensed by ARM.

The PrimeCell SMC is an AMBA slave module and connects to the Advanced High-performance Bus (AHB).

The PrimeCell SMC supports:

- static memory-mapped devices including RAM, ROM, flash, and burst ROM
- asynchronous page mode read operation in non-clocked memory subsystems
- asynchronous burst mode read access from burst mode ROM and flash devices
- 8, 16, and 32-bit wide external memory data paths
- little-endian and big-endian memory architectures
- AHB burst transfers
- independent configuration for up to eight memory banks, each up to 64Mb
- programmable wait states (up to 31)
- programmable bus turnaround cycles (up to 15)
- programmable output enable and write enable delays (up to 15)
- write enable and byte lane select outputs for use with 32, 16, or 8-bit SRAM devices
- independent byte lane control for each memory bank
- external asynchronous wait control
- configurable size at reset for boot memory bank using external control pins
- system testing using externally applied TIC vectors through built-in TIC AMBA master block
- access port allows a future device access to external bus.

3 FEATURES

The PrimeCell SMC driver supports independent configuration for up to eight memory banks with the following programmable parameters:

- external memory width, 8, 16, or 32-bit
- burst mode operation
- write protection
- chip select polarity
- external wait control enable
- external wait polarity
- write WAIT states for static RAM devices
- read WAIT states for static RAM and ROM devices
- initial and subsequent burst read WAIT state for burst ROM devices
- read byte lane enable control
- bus turn-around (idle) cycles
- output enable and write enable output delays.

4 DRIVER DESCRIPTION

4.1 Overview

The SMC driver contains the set of functions that allow to configure up to eight memory banks, read their configuration, disable and enable write protection, and define status of error flags for each memory bank.

4.2 Driver Configuration

The SMC driver can be configured through the macro apSMC_BANK7_BOOT in system description file **`applatfm.h`**.

If apSMC_BANK7_BOOT is defined in **`applatfm.h`**, which indicates that the system boots from Bank 7, apSMC_Initialize() function will not set up default reset values for Bank 7, otherwise the default configuration parameters for this memory bank will be programmed.

4.3 Driver Initialisation

- Call apSMC_Initialize().

4.4 Configuration of the Memory Banks

- Call apSMC_MemBankConfigSet() to configure the specified memory bank.

4.5 Reading Configuration of the Memory Banks

- Call apSMC_MemBankConfigGet() to read configuration of the specified memory bank.

4.6 Write Protection of Memory Banks

- Call apSMC_WriteEnable() to enable write operations for the specified memory bank;
- Call apSMC_WriteDisable() to disable write operations for the specified memory bank.

4.7 Reading Error Status Flags

- Call apSMC_ErrorGet() to read and clear the error status flags for the specified memory bank.

5 DETAILS

- Peak performance obtained in tests: N/A
- ROM size (RO code + RO data) (release build): 840 Bytes
- RAM size (ZI data) per instance (release build): 8 Bytes
- Does this driver support multiple PrimeCells?: YES
- Can these be detected at run-time? NO
- Does this driver support both endian-nesses?[1] YES
- Does this driver support Thumb?[2] YES

6 RELEASED FILES

`PL092-ISPC-1000`	This API document
`PL092–TREP-1000`	Test Results document
`apSMC/apSMC.h`	Public header file
`apSMC/SMC.h`	Private header file
`apSMC/SMC.c`	Source code
`apSMC/SMCtst.c`	Test code

[1] To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

[2] The driver may include ARM code, but this must interwork with a Thumb build.

7 PUBLIC API

7.1 Type Definitions

7.1.1 apSMC_eRBLE

This specifies options for read byte lane enable.

Declaration

```
typedef enum apSMC_eRBLE
```

Elements

apSMC_RBLE_HI	nSMBLS[3:0] all deasserted HIGH during system reads from external memory (default at reset)
apSMC_RBLE_LO	nSMBLS[3:0] all deasserted LOW during system reads from external memory

7.1.2 apSMC_eWaitPol

This specifies options for polarity of the external wait input for activation.

Declaration

```
typedef enum apSMC_eWaitPol
```

Elements

apSMC_WAITPOL_LO	The SMWAIT signal is active LOW (default at reset)
apSMC_WAITPOL_HI	The SMWAIT signal is active HIGH

7.1.3 apSMC_eWaitEn

This specifies options for external memory controller wait signal enable.

Declaration

```
typedef enum apSMC_eWaitEn
```

Elements

apSMC_WAITEN_NO	SMC is not controlled by the external wait signal (default at reset)
apSMC_WAITEN_YES	SMC looks for the external wait input signal SMWAIT

7.1.4 apSMC_eCSPol

This specifies options for chip select polarity for each bank.

Declaration

```
typedef enum apSMC_eCSPol
```

Elements

apSMC_CSPOL_LO	Active LOW SMCS (default at reset)
apSMC_CSPOL_HI	Active HIGH SMCS

7.1.5 apSMC_eWP

This specifies options for write protection.

Declaration

```
typedef enum apSMC_eWP
```

Elements

apSMC_WP_NO	No write protection (default at reset)
apSMC_WP_YES	Device is write protected

7.1.6 apSMC_eBM

This specifies options for burst mode.

Declaration

```
typedef enum apSMC_eBM
```

Elements

apSMC_BM_NO	Non-burst memory devices (default at reset)
apSMC_BM_YES	Burst ROM memory

7.1.7 apSMC_eMW

This specifies options for memory width.

Declaration

```
typedef enum apSMC_eMW
```

Elements

apSMC_MW_8BIT	8-bit
apSMC_MW_16BIT	16-bit
apSMC_MW_32BIT	32-bit

7.1.8 apSMC_eError

This specifies the general errors reported by this module.
Errors will be returned typecast to the general error type apError.

Declaration

```
typedef enum apSMC_eError
```

Elements

apERR_SMC_INVALIDBANK	Invalid Bank number
apERR_SMC_INVALIDREADWAITSTATES	Invalid value of wait states for read access
apERR_SMC_INVALIDWRITEWAITSTATES	Invalid value of wait states for write access
apERR_SMC_INVALIDIDLECYCLES	Invalid value of duty or turn-around cycles
apERR_SMC_INVALIDOUTPUTENDELAY	Invalid value of output enable assertion delay
apERR_SMC_INVALIDWRITEENDELAY	Invalid value of write enable assertion delay

7.1.9 apSMC_sData

The data structure to hold parameters for initialisation of each memory block.

Declaration

```
struct apSMC_sData
{
    apSMC_eRBLE    eRBLE;
    apSMC_eWaitPol eWaitPol;
    apSMC_eWaitEn  eWaitEn;
    apSMC_eCSPol   eCSPol;
    apSMC_eWP      eWriteProtect;
    apSMC_eBM      eBurstMode;
    apSMC_eMW      eMemoryWidth
    UBYTE8         ReadAccessCycles;
    UBYTE8         WriteAccessCycles;
    UBYTE8         IdleTurnAroundCycles;
    UBYTE8         OutputEnAssertDelay;
    UBYTE8         WriteEnAssertDelay;
}
```

Elements

eRBLE	Read byte lane enable
eWaitPol	Polarity of the external wait signal for activation
eWaitEn	External memory controller wait signal enable
eCSPol	Chip select polarity
eWriteProtect	Write protection
eBurstMode	Burst mode
eMemoryWidth	Memory width, 8, 16, or 32-bit
ReadAccessCycles	Wait state 1 in clock cycles, values in range 0-31 (defaults to 31 at reset)
WriteAccessCyclest	Wait state 2 in clock cycles, values in range 0-31 (defaults to 31 at reset)
IdleTurnAroundCycles	Idle or turn-around in clock cycles, values in range 0-15 (defaults to 15 at reset)
OutputEnAssertDelay	Output enable assertion delay in clock cycles, values in range 0-15 (defaults to 0 at reset)
WriteEnAssertDelay	Write enable assertion delay in clock cycles, values in range 0-15 (defaults to 0 at reset)

7.2 External Functions

7.2.1 apSMC_Initialize

Initialises the SMC device.

Declaration

```
PUBLIC void apSMC_Initialize ( apOS_SMC_oId                       oId,
                               apOS_System_eBaseAddress           eBase,
                               UWORD32                            Interrupts,
                               CONST apOS_INT_oInterruptSource *  pSources,
                               apSMC_sInitialData              *  pInitial )
```

Parameters

oId	SMC device number to be initialised
eBase	Base address of SMC device control registers
Interrupts	Ignored (there are no interrupts for the controller)
pSources	Not used, pass as aNULL
pInitial	Not used, pass as aNULL

Return Value

None

Remarks

This function configures all memory banks with default reset parameters and marks all banks as non-initialised by writing zero value in initialisation state variable. If macro apSMC_BANK7_BOOT is defined in **`applatfm.h`**, default parameters for Bank 7 are not set.

7.2.2 apSMC_InitGet

Returns the current value of initialisation state variable.

Declaration

```
PUBLIC UBYTE8 apSMC_InitGet (apOS_SMC_oId oId )
```

Parameters

oId	SMC device number to get initialisation state

Return Value

Initialisation flags for each memory bank

Remarks

Bit is set if the corresponding memory bank was initialised.

7.2.3 apSMC_StateSizeGet

Retrieves the size required for storage of the driver state data.

Declaration

```
PUBLIC UWORD32 apSMC_StateSizeGet ( void )
```

Parameters

None

Return Value

Size (in bytes) required

Remarks

This function is required if apOS_NO_STATIC_STATE is defined as TRUE.

7.2.4 apSMC_MemBankConfigSet

Configures the specified memory bank.

Declaration

```
PUBLIC apError apSMC_MemBankConfigSet ( apOS_SMC_oId        oId,
                                        UWORD32             MemBank,
                                        CONST apSMC_sData * pBankData )
```

Parameters

oId	Id number of SMC device
MemBank	The bank number to be configured
pBankDatak	Pointer to a block of data for configuring the memory bank

Return Value

apERR_NONE	Configuration parameters set, no errors
apERR_SMC_INVALIDBANK	Parameters not set because of the invalid value of bank number
apERR_SMC_INVALIDREADWAITSTATES	Parameters not set because of the invalid value of wait states for read access
apERR_SMC_INVALIDWRITEWAITSTATES	Parameters not set because of the invalid value of wait states for write access
apERR_SMC_INVALIDIDLECYCLES	Parameters not set because of the invalid value of duty or turn-around cycles
apERR_SMC_INVALIDOUTPUTENDELAY	Parameters not set because of the invalid value of output enable assertion delay
apERR_SMC_INVALIDWRITEENDELAY	Parameters not set because of the invalid value of write enable assertion delay

Remarks

This function writes the initialisation values in registers for specified bank and sets the corresponding bit in the initialisation state variable.

7.2.5 apSMC_MemBankConfigGet

Reads configuration of the specified memory bank.

Declaration

```
PUBLIC apError apSMC_MemBankConfigGet ( apOS_SMC_oId        oId,
                                        UWORD32             MemBank,
                                        apSMC_sData * CONST pBankData )
```

Parameters

oId	Id number of SMC device
MemBank	The bank number to read configuration
pBankDatak	Pointer to a block of data for reading configuration of the memory bank

Return Value

apERR_NONE	Configuration parameters read, no errors
apERR_SMC_INVALIDBANK	Parameters not read because of the invalid value of bank number

Remarks

This function reads values of bank registers and decodes them into configuration parameters.

7.2.6 apSMC_WriteEnable

Enables write operations in external memory.

Declaration

```
PUBLIC apError apSMC_WriteEnable ( apOS_SMC_oId oId,
                                   UWORD32      MemBank )
```

Parameters

oId	Id number of SMC device
MemBank	The bank number to be write enabled

Return Value

apERR_NONE	Memory bank write enabled, no errors
apERR_SMC_INVALIDBANK	Memory bank not write enabled because of the invalid value of bank number

Remarks

After calling this function the specified memory bank can be written to.

7.2.7 apSMC_WriteDisable

Disables write operations in external memory.

Declaration

```
PUBLIC apError apSMC_WriteDisable ( apOS_SMC_oId oId,
                                    UWORD32      MemBank )
```

Parameters

oId	Id number of SMC device
MemBank	The bank number to be write disabled

Return Value

apERR_NONE	Memory bank write disabled, no errors
apERR_SMC_INVALIDBANK	Memory bank not write disabled because of the invalid value of bank number

Remarks

After calling this function the specified memory bank is write protected.

7.2.8 apSMC_ErrorGet

Reads the error status flags for the specified memory bank.

Declaration

```
PUBLIC apError apSMC_ErrorGet( apOS_SMC_oId oId,
                               UWORD32      MemBank,
                               BOOL * CONST pWaitToutErr,
                               BOOL * CONST pWriteProtErr,
                               BOOL * CONST pHSizeErr )
```

Parameters

oId	Id number of SMC device
MemBank	The bank number to read error status flags
pWaitToutErr	Pointer to the variable that is set TRUE if there was external wait timeout error
pWriteProtErr	Pointer to the variable that is set TRUE if there was write protection error
pHSizeErr	Pointer to the variable that is set TRUE if there was bus transfer size error

Return Value

apERR_NONE	Error status flags read, no errors
apERR_SMC_INVALIDBANK	Error status flags not read because of the invalid value of bank number

Remarks

This function reads and clears error status bits of status register for the specified memory bank.